

AI 데이터 분석 · 10일차

딕셔너리

번호가 아니라 이름표로 찾는다

키·값 · get · in · keys · values · items · update · zip · 중첩 딕셔너리

교안 06_02(딕셔너리) 전체 + 실습 1~8 정리

기본 자료형 4형제의 마지막 — 그리고 실무에서 가장 많이 쓰는 하나

2026-08-03 AI 데이터 분석 교육 정리 (10일차)

학습자 Minho · 국립금오공과대학교 컴퓨터공학부

이 책을 읽는 법

리스트·튜플·셋을 배우고 마지막 하나가 남았습니다. 딕셔너리입니다. 앞의 셋은 모두 "몇 번째"로 값을 찾았지만 딕셔너리는 "무슨 이름"으로 찾습니다. 이 차이 하나가 실무에서 딕셔너리를 가장 많이 쓰이게 만듭니다. 설비 데이터는 애초에 온도=78, 진동=0.5처럼 이름표가 붙은 형태로 들어오기 때문입니다.

딕셔너리 {키: 값} 메서드 get·items·update 키워드 del 연산자 in 내장함수 zip·len·sum

연결

오늘 수업에서 강사님이 여러 번 강조한 문장이 있습니다 — "딕셔너리 하나만 쓰이는 사례가 거의 없고, 리스트 하나만 쓰이는 사례도 거의 없다." 실무 데이터는 [{...}, {...}, {...}] 즉 딕셔너리들을 담은 리스트 형태입니다. 엑셀 표를 떠올리면 정확합니다 — 한 행이 딕셔너리 하나, 표 전체가 리스트. 이 구조가 오늘의 도착점이자 pandas의 출발점입니다.

목차

	장 / STEP	무엇을 배우나
CHAPTER 0	오늘의 지도 — 왜 딕셔너리인가	리스트의 한계 · 자료형 4형제 완성
STEP 1	딕셔너리 만들기	키:값 · 키의 제약 · 스네이크 케이스
STEP 2	값 꺼내기 — []와 get	KeyError · 기본값 · in
STEP 3	추가 · 수정 · 삭제	같은 문법 · del · = 읽는 법
STEP 4	keys · values · items	뷰 객체 · items가 필수인 이유
STEP 5	items 순회와 두 딕셔너리 비교	임계값 판정 패턴
STEP 6	update 와 zip	편하지만 사고가 나는 두 기능
STEP 7	딕셔너리 리스트와 중첩	실무에서 가장 많이 보는 구조
STEP 8	내 코드 점검 — 16_dictionary.py	기본값 0이 만든 오경보
STEP 9	실습 1~4 복기	제출 코드 점검
STEP 10	실습 5~8	중복 버그 · 미완성 3건 완성
부록 A·B·C	치트시트 · 오류 사전 · 예고	한 장 요약 · 함정 모음

오늘의 지도 — 왜 딕셔너리인가

번호로는 담을 수 없는 정보가 있다

강사님이 딕셔너리를 꺼내기 전에 리스트로 먼저 문제를 보여주셨습니다. 데이터 분석 수업의 임원진을 리스트로 적으면 이렇게 됩니다.

리스트로 적으면

```
data_class_list = ["태구", "수진", "영준", "민지", "현우"]
```

이 리스트를 다른 사람에게 전화로 불러준다고 해봅시다. "태구, 수진, 영준..." 을 받아 적은 사람이 누가 반장이고 누가 총무인지 알 수 있을까요? 알 수 없습니다. 첫 번째가 반장이라는 건 **우리끼리의 암묵적 약속**일 뿐, 데이터 어디에도 적혀 있지 않습니다.

개념

강사님 표현으로는 **"무의미한 순서가 있을 뿐"**입니다. 리스트가 메모리에 저장하는 정보는 값과 순서, 그 둘뿐입니다. **"이 값이 무엇인지"**는 저장할 방법이 없습니다. 그래서 별도로 설명을 덧붙여야 하고, 설명이 유실되면 데이터도 함께 무의미해집니다.

딕셔너리는 그 설명을 데이터 안에 함께 넣습니다.

딕셔너리로 적으면

```
data_class_dict = {  
    "반장": "태구",  
    "부반장": "수진",  
    "총무": "영준",  
    "서기": "민지",  
    "회계": "현우",  
}
```

이제는 이 데이터만 받아도 누가 무슨 역할인지 알 수 있습니다. **역할까지 부여할 수 있는 자료구조** — 이것이 딕셔너리를 쓰는 이유입니다.

자료형 4형제 — 오늘로 완성

자료형	괄호	찾는 방법	순서	중복	대표 용도
리스트	[]	번호(인덱스)	있음	허용	계속 추가·수정되는 측정값
튜플	()	번호(인덱스)	있음	허용	바뀌면 안 되는 짝
셋	{ }	찾을 수 없음	없음	제거	종류 세기 · 목록 비교
딕셔너리	{키: 값}	이름(키)	넣은 순서	키는 유일	이름표가 붙은 모든 데이터

주의

셋과 딕셔너리가 같은 **중괄호**를 씁니다. 구분은 안에 무엇이 들어 있느냐입니다 — 값만 나열하면 셋, 키: 값 형태면 딕셔너리입니다. 그리고 어제 배운 대로 **빈 중괄호** {}는 딕셔너리입니다. 딕셔너리가 파이썬에 먼저 자리 잡았기 때문에 {}를 선점했고, 나중에 온 셋이 set()이라는 별도 표기를 쓰게 되었습니다.

용어 3초 판별법 — 오늘 배운 것에 적용하면

① 앞에 점(.)이 있나?	② 뒤에 괄호()가 붙나?	결론	오늘의 예
없다	붙는다	내장 함수	len(d) · dict() · zip() · sum() · sorted(d)
있다	붙는다	메서드	d.get() · d.keys() · d.items() · d.update()
없다	안 붙는다	키워드·연산자	del · in · for · if · =
없다	대괄호가 붙는다	문법(인덱싱)	d["모터온도"] · d["A"]["B"]

개념

오늘 배우는 딕셔너리 **메서드**는 다섯 개입니다 — get · keys · values · items · update. 교안에 나오는 것은 이게 전부이고, pop · clear · copy · setdefault 같은 다른 메서드는 등장하지 않습니다. 대신 **메서드가 아닌 것들**이 잔뜩 섞여 나오는 게 오늘의 혼란 지점입니다. del은 키워드, in은 연산자, len · zip은 내장 함수, d[키] = 값은 문법입니다. 점이 찍혀 있지만 보면 즉시 갑니다.

오늘의 흐름



STEP 1

딕셔너리 만들기

키와 값을 콜론으로 잇는다

자료형 dict 이름표로 값을 찾는 자료구조

정의 중괄호 안에 키: 값 쌍을 쉼표로 나열합니다. 키는 이름표, 값은 그 이름표가 가리키는 데이터입니다.

형태 {키1: 값1, 키2: 값2, ...}

```
sensors = {"모터온도": 78, "진동": 0.5}

print(sensors)
print(type(sensors))
print(len(sensors))          # 키-값 조합이 몇 쌍인지
```

- ▶ {'모터온도': 78, '진동': 0.5}
- ▶ <class 'dict'>
- ▶ 2

왜 쓰나 센서 데이터가 원래 이 모양이기 때문입니다. 현장에서 오는 데이터는 78, 0.5처럼 숫자만 오지 않고 "무엇의 값인지"가 붙어서 옵니다. 그 형태를 그대로 담을 수 있는 자료형이 딕셔너리입니다.

응용 값의 자료형은 섞여도 됩니다. 위 예시에서 78은 int, 0.5는 float입니다. 문자열도 리스트도 들어갈 수 있습니다. 하나의 센서가 이름·수치·상태를 함께 들고 다닐 수 있다는 뜻입니다.

```
sensor = {
    "센서 이름": "motor_vibration",    # 문자열
    "진동": 4.8,                        # 실수
    "가동": True,                      # 불리언
    "최근 3회": [4.8, 5.1, 4.9],       # 리스트
}
```



```
for key in sensor:
    print(f"{key:10} {type(sensor[key]).__name__:6} {sensor[key]}")
```

- ▶ 센서 이름 str motor_vibration
- ▶ 진동 float 4.8
- ▶ 가동 bool True
- ▶ 최근 3회 list [4.8, 5.1, 4.9]

개념

len(딕셔너리)는 **키-값 쌍의 개수**를 셉니다. 같은 len이라도 대상에 따라 세는 것이 달라집니다 — 문자열은 글자 수, 리스트·튜플은 요소 수, 셋은 종류 수, 딕셔너리는 쌍의 수입니다.

문법 키가 될 수 있는 것 값은 자유, 키는 제약이 있다

정의 키는 **변하지 않는 값**만 가능합니다. 문자열·숫자·튜플은 되지만 리스트는 안 됩니다.

```
ok1 = {"모터온도": 78}      # 문자열 키 — 가장 흔함
ok2 = {1: "첫 번째"}       # 숫자 키
ok3 = {("A동", 3): 78}     # 튜플 키 — 좌표를 키로
```

```
# ng = [{"A동", 3]: 78}
# TypeError: unhashable type: 'list'
```

- ▶ {'모터온도': 78}
- ▶ {1: '첫 번째'}
- ▶ {('A동', 3): 78}

왜 쓰나 키는 값을 찾아가는 **주소** 역할을 합니다. 주소가 도중에 바뀌면 저장해 둔 값을 다시 찾을 수 없습니다. 그래서 파이썬은 바뀔 수 있는 자료형(리스트·셋·딕셔너리)을 키로 쓰지 못하게 막았습니다.

응용 어제 셋을 배울 때 나온 `unhashable type` 오류가 여기서 똑같이 나옵니다. **셋의 원소가 될 수 있는 것 = 딕셔너리의 키가 될 수 있는 것**입니다. 튜플이 "바뀌지 않아 딕셔너리 키로도 쓸 수 있다"고 배운 것이 바로 이 대목입니다.

주의

실무에서는 **키를 문자열로 두는 경우가 압도적**입니다. 값은 문자열이든 정수든 실수든 자유롭게 섞이지만, 키만큼은 이름표이므로 사람이 읽을 수 있는 문자열이 자연스럽습니다.

규칙 스네이크 케이스 파이썬의 이름 짓는 관례

정의 단어를 밑줄(_)로 잇고 전부 소문자로 쓰는 방식. 파이썬이 공식적으로 권장하는 변수·함수 이름 규칙입니다.

```
# 스네이크 케이스 — 파이썬 권장
data_class_dict = {}
sensor_value = 78
motor_temp_limit = 90

# 카멜 케이스 — 자바·자바스크립트에서 흔함
dataClassDict = {}
sensorValue = 78
```

왜 쓰나 문법상으로는 둘 다 동작합니다. 하지만 강사님이 시간을 들여 설명하신 이유가 있습니다 — **협업할 때 코드를 주고받으면 섞인 표기가 곧 혼란**이 되기 때문입니다. 파이썬 공식 스타일 가이드(PEP 8)가 스네이크 케이스를 권장하므로, 파이썬을 쓰는 동안은 여기에 맞추는 것이 정석입니다.

응용 이름이 왜 그렇게 불리는지 알면 안 잊힙니다. **스네이크(뱀)**는 _가 바닥에 깔려 뱀처럼 이어져 보인다고 해서, **카멜(낙타)**은 중간중간 대문자가 낙타 등처럼 툭 튀어나온다고 해서 붙은 이름입니다. 참고로 첫 글자까지 대문자로 쓰는 `DataClass` 형태는 파스칼 케이스라고 하며, 파이썬에서는 **클래스 이름**에 씁니다.

```
# 파이썬에서 쓰는 세 가지 관례
sensor_value = 78          # 스네이크 — 변수·함수
LIMIT_TEMPERATURE = 90    # 대문자 스네이크 — 상수 (바뀌지 않을 값)
class SensorReader:        # 파스칼 — 클래스 (아직 안 배움)
    pass
```

개념

상수를 대문자로 쓰는 관례는 이미 써 왔습니다. 종합 실습에서 `LIMIT = 100`이라고 쓴 것이 그것입니다. 파이썬이 강제하는 것은 아니지만, 대문자를 보면 **"이건 바뀌지 않는 기준값이구나"**를 코드만 보고 알 수 있게 하는 신호입니다.

왜 파이썬으로 데이터 분석을 하나

수업 중반에 강사님이 딕셔너리를 실행해 보이면서 곁들인 이야기입니다. 문법과 직접 관계는 없지만 **왜 이 언어를 배우는지**에 대한 답이라 정리해 둡니다.

	컴파일 방식 (C · 자바)	인터프리터 방식 (파이썬)
번역 시점	전체를 한 번에 기계어로 번역 후 실행	한 줄씩 그때그때 번역하며 실행
비유	책 한 권을 통째로 번역	공연장의 동시통역
실행 속도	빠름	상대적으로 느림
수정 후 확인	전체를 다시 번역해야 함	바로 실행해 확인
어울리는 일	한 번 만들어 오래 돌리는 프로그램	자주 고쳐야 하는 데이터 분석

설비 적용

강사님이 든 근거가 현장 그대로입니다 — "데이터들이 그때그때 형식이 약간씩 다를 때가 있다." 이번 달 데이터에는 있던 컬럼이 다음 달엔 빠지고, 센서가 추가되고, 단위가 바뀝니다. 그럴 때마다 코드를 고쳐 바로 돌려봐야 하는데, 전체를 다시 번역해야 한다면 그 시간이 감당이 안 됩니다. **파이썬이 문법이 쉬워서 쓰는 게 아니라, 잦은 수정에 유리해서** 데이터 분석의 언어가 된 것입니다.

만든 딕셔너리를 확인하는 습관

강사님이 딕셔너리를 만들 때마다 빠짐없이 하신 동작이 있습니다 — **바로 출력해서 눈으로 확인하기**입니다. 코드가 길어지기 전에 지금 만든 것이 의도한 모양인지 확인하는 습관은 나중에 디버깅 시간을 크게 줄여 줍니다.

다섯 줄 점검

```
sensors = {"센서 이름": "보일러", "모터온도": 78, "진동": 0.5}
```

```
print(sensors)           # ① 전체 모양
print(type(sensors))     # ② 정말 딕셔너리가 맞는지
print(len(sensors))      # ③ 쌍이 몇 개인지
print(list(sensors.keys())) # ④ 키 이름이 의도대로인지
print(type(sensors["모터온도"])) # ⑤ 값의 자료형까지
```

```
▶ {'센서 이름': '보일러', '모터온도': 78, '진동': 0.5}
▶ <class 'dict'>
▶ 3
▶ ['센서 이름', '모터온도', '진동']
▶ <class 'int'>
```

주의

④가 특히 중요합니다. 키 이름에 **공백**이나 **오타**가 **섞이면 조회가 전부 실패**하는데, `KeyError`가 날 때까지 알아채기 어렵습니다. "모터 온도"와 "모터온도"는 파이썬에게 완전히 다른 키입니다. 오늘 수업 코드에도 "모터 온도"(공백 있음)와 "모터온도"(공백 없음)가 파일마다 섞여 있는데, 두 딕셔너리를 대조할 일이 생기면 이게 그대로 사고가 됩니다. **키 이름 규칙을 처음부터 통일**해 두는 편이 안전합니다.

그리고 파이썬 파일을 실행할 때 **저장을 먼저** 해야 합니다. 저장하지 않고 실행하면 이전 코드가 돌아가서 "왜 안 바뀌지?"로 시간을 버리게 됩니다. 단축키는 윈도우 `Ctrl + S`, 맥 `Cmd + S`입니다.

TERMINAL

```
# VS Code 터미널에서 실행
python 16_dictionary.py          # 윈도우
python3 16_dictionary.py        # 맥 (2 버전과 섞여 있을 수 있어 3을 명시)

python --version                 # 지금 쓰는 버전 확인
```

첫 딕셔너리 — 수업에서 만든 임원진 데이터

CHAPTER 0에서 본 임원진 딕셔너리를 실제로 다뤄 보면 오늘 배울 것이 한 번에 미리 보입니다. 아직 안 배운 문법이 섞여 있으니 **지금은 흐름만** 눈에 담아 두면 됩니다.

딕셔너리 한 바퀴

```
data_class_dict = {"반장": "태구", "부반장": "수진", "총무": "영준", "회계": "현우"}

print("반장은?", data_class_dict["반장"])          # 이름표로 조회
data_class_dict["서기"] = "민지"                  # 없는 키 → 추가
print("임원 수:", len(data_class_dict))             # 쌍의 개수

for role, name in data_class_dict.items():         # 역할과 이름을 함께
    print(f" {role:4} {name}")
```

- ▶ 반장은? 태구
- ▶ 임원 수: 5
- ▶ 반장 태구
- ▶ 부반장 수진
- ▶ 총무 영준
- ▶ 회계 현우
- ▶ 서기 민지

개념

리스트였다면 `data_class_list[0]`이 반장이라는 걸 **코드 밖에서 따로 알고 있어야** 했습니다. 딕셔너리는 `data_class_dict["반장"]`이라고 쓰는 순간 코드가 스스로 설명합니다. 몇 달 뒤에 이 코드를 다시 열어도, 다른 사람이 받아 봐도 해석이 필요 없습니다. **좋은 자료구조는 주석을 줄인다는** 말이 이런 뜻입니다.

STEP 2

값 꺼내기 — [] 와 get

없는 키를 물었을 때가 갈림길

딕셔너리에서 값을 꺼내는 방법은 두 가지입니다. 대괄호와 get입니다. 키가 **확실히 있을 때는 둘이 똑같이 동작**하지만, 없는 키를 물었을 때 완전히 달라집니다. 이 차이가 실무에서 프로그램이 죽느냐 마느냐를 가릅니다.

문법 딕셔너리[키] 대괄호로 값 꺼내기

정의 대괄호 안에 키를 넣으면 그 값이 나옵니다. **없는 키를 넣으면 KeyError로 프로그램이 멈춥니다.**

형태 딕셔너리[키]

```
sensors = {"모터온도": 78, "진동": 0.5}

print(sensors["모터온도"])
print(sensors["진동"])

# print(sensors["유량"])
# KeyError: '유량'

> 78
> 0.5
```

왜 쓰나 리스트가 lst[0]이었다면 딕셔너리는 d["모터온도"]입니다. **위치가 아니라 이름으로** 찾는다는 것만 다릅니다. 키가 반드시 있다고 보장되는 상황에서는 가장 짧고 명확한 방법입니다.

응용 값이 리스트나 딕셔너리면 대괄호를 이어서 씁니다. 어제 중첩 튜플에서 t[2][0]을 쓴 것과 똑같은 원리입니다.

```
equipment = {
    "Pump_B": {"status": "경고", "temperature": 68.1},
}

print(equipment["Pump_B"])           # 안쪽 딕셔너리 전체
print(equipment["Pump_B"]["temperature"]) # 그 안의 값

> {'status': '경고', 'temperature': 68.1}
> 68.1
```

자주 하는 실수

`KeyError`는 **실행이 그 자리에서 멈추는** 오류입니다. 센서 하나에서 값이 안 왔다는 이유로 전체 분석이 중단되면 곤란하죠. 그래서 키의 존재가 확실하지 않을 때는 아래 `get`이나 `in`을 씁니다.

메서드 `.get()` 없는 키여도 멈추지 않는 조회

정의 키가 있으면 값을, 없으면 `None`을 돌려줍니다. 두 번째 인자로 **기본값**을 주면 `None` 대신 그 값이 나옵니다.

형태 `딕셔너리.get(키)` · `딕셔너리.get(키, 기본값)`

```
sensors = {"모터온도": 78, "진동": 0.5}

print(sensors.get("모터온도"))    # 있는 키
print(sensors.get("유량"))        # 없는 키 → None
print(sensors.get("유량", 0))     # 없으면 0
print(sensors.get("유량", -1))    # 없으면 -1
```

```
▶ 78
▶ None
▶ 0
▶ -1
```

왜 쓰나 강사님이 든 상황이 정확합니다 — **센서가 고장 나서 데이터가 안 올 수도 있습니다**. 그때 프로그램이 죽어버리면 나머지 센서 분석까지 전부 날아갑니다. `get`은 "없으면 없는 대로 넘어가자"를 문법으로 만들어 둔 것입니다.

응용 기본값을 무엇으로 정하느냐가 실무의 판단입니다. **0을 기본값으로 쓰면 위험한 경우가 많습니다** — 0도 실제로 측정될 수 있는 값이라, 나중에 결과를 보면 "측정값이 0이었다"인지 "데이터가 없었다"인지 구분이 안 됩니다. 그래서 측정값이 절대 가질 수 없는 값(예: -1)을 기본값으로 두는 관례가 있습니다.

```

readings = {"모터온도": 78, "진동": 0.5}

# 나쁜 예 — 0이 실제 값인지 결측인지 구분 불가
print("유량:", readings.get("유량", 0))

# 좋은 예 — 음수는 측정값이 될 수 없으므로 결측 신호로 쓴다
flow = readings.get("유량", -1)
if flow < 0:
    print("유량: 데이터 없음 (센서 점검 필요)")
else:
    print("유량:", flow)

```

```

▶ 유량: 0
▶ 유량: 데이터 없음 (센서 점검 필요)

```

자주 하는 실수

기본값 선택을 잘못하면 **오류 없이 결과만 틀립니다**. STEP 8에서 오늘 수업 코드에 실제로 이 일이 일어난 사례를 다룹니다.

연산자 `in` 키가 있는지 먼저 확인

정의 키 `in` 딕셔너리는 그 **키가** 있으면 `True`입니다. 값이 아니라 키를 봅니다.

형태 키 `in` 딕셔너리 · 값 `in` 딕셔너리.values()

```

sensors = {"모터온도": 78, "진동": 0.5}

print("진동" in sensors)      # 키로 확인
print("면적" in sensors)
print(0.5 in sensors)        # 값으로는 못 찾는다
print(0.5 in sensors.values()) # 값을 보려면 values()

```

```

▶ True
▶ False
▶ False
▶ True

```

왜 쓰나 `get`으로 기본값을 넘기는 방법과, `in`으로 먼저 확인하는 방법 두 가지가 있습니다. 강사님 표현으로 **"기본값을 무조건 넣는 분도 있고, 키부터 따져보고 있을 때만 계산하는 분도 있다"** — 상황에 따라 고르면 됩니다.

응용 둘의 차이는 "없을 때 무엇을 하고 싶은가"입니다. 없어도 계산은 이어가야 하면 `get`의 기본값이 편하고, 없으면 아예 다른 처리를 해야 하면 `in`으로 갈라내는 편이 명확합니다.

```
sensors = {"모터온도": 78, "진동": 0.5}

# 방법 1 - get에 기본값 (계산을 이어간다)
total = sensors.get("모터온도", 0) + sensors.get("유량", 0)
print("합계:", total)

# 방법 2 - in으로 갈라낸다 (없으면 다른 처리)
for key in ["진동", "면적"]:
    if key in sensors:
        print(f"{key}: {sensors[key]}")
    else:
        print(f"{key}: 미설치 센서 — 집계 제외")
```

▶ 합계: 78
▶ 진동: 0.5
▶ 면적: 미설치 센서 — 집계 제외

개념

`in`은 딕셔너리의 **키**를 봅니다. 리스트에서 `in`이 요소를 보고, 셋에서 원소를 보던 것과 대비하면 헷갈리지 않습니다 — 딕셔너리에서 "요소"에 해당하는 게 키이기 때문입니다.

STEP 3

추가 · 수정 · 삭제

같은 문법이 두 가지 일을 한다

문법 `딕셔너리[키] = 값` 추가와 수정을 동시에

정의 없는 키면 새로 추가되고, 있는 키면 값이 덮어씌웁니다. 문법이 하나입니다.

형태 `딕셔너리[키] = 값`

```
sensors = {"모터온도": 78}

sensors["펌프압력"] = 95      # 없는 키 → 추가
sensors["유량"] = 42         # 없는 키 → 추가
print(sensors)

sensors["모터온도"] = 81      # 있는 키 → 수정
print(sensors)
```

- ▶ {'모터온도': 78, '펌프압력': 95, '유량': 42}
- ▶ {'모터온도': 81, '펌프압력': 95, '유량': 42}

왜 쓰나 리스트는 추가가 `append`, 수정이 `lst[0] = x`로 나뉘었습니다. 딕셔너리는 하나로 통합되어 있습니다.

강사님이 이 점을 특히 강조하셨는데, 실시간으로 들어오는 센서 데이터를 다룰 때 "이 키가 처음인지 아닌지" 신경 쓰지 않아도 되기 때문입니다.

응용 그래서 빈 딕셔너리로 시작해 채워 나가는 패턴이 성립합니다. 강사님 말씀대로 처음부터 완성된 딕셔너리로 시작해 끝까지 그대로 쓰는 경우는 거의 없고, 대부분 실행 중에 데이터가 들어오면서 쌓입니다.

```

readings = {}                                # 빈 딕셔너리로 시작

raw = [("모터온도", 78), ("진동", 0.5), ("모터온도", 81)]
for name, value in raw:                      # 어제 배운 튜플 리스트 순회
    readings[name] = value                    # 처음이면 추가, 있으면 갱신

print(readings)
print("최종 모터온도:", readings["모터온도"])

```

```

▶ {'모터온도': 81, '진동': 0.5}
▶ 최종 모터온도: 81

```

개념

위 예시에서 모터온도가 두 번 들어왔지만 결과는 하나이고 값은 **나중 것(81)**입니다. 딕셔너리는 같은 키를 중복해서 가질 수 없으므로 항상 **최신 값만** 남습니다. 센서 현재값 관리에 딱 맞는 성질입니다.

= 를 읽는 법

강사님이 코딩 입문자를 위해 따로 설명한 팁입니다. 등호를 볼 때는 **오른쪽부터** 읽으라는 것입니다.

읽는 순서

```
sensors["펌프압력"] = 95
```

```

#           ② ←———— ①
#   왼쪽에 집어넣는다   오른쪽 값을 먼저 본다

```

개념

오른쪽에는 값이든 계산식이든 **결과가 하나 만들어지는 것**이 옵니다. 그 결과를 왼쪽에 **집어넣는 것**이 등호의 역할입니다. 수학의 "같다"가 아닙니다. 실제로 일부 프로그래밍 언어는 이 동작을 화살표(←)로 표기하기도 합니다. 강사님은 전문 용어인 "할당 연산자"보다 **"집어넣는다"**는 표현을 쓰겠다고 하셨는데, 초반에는 그 편이 훨씬 덜 헷갈립니다.

키워드 `del` 키-값 쌍을 통째로 삭제

정의 `del` 딕셔너리[키]로 그 키와 값을 **함께** 지웁니다. 값만 지우는 것이 아닙니다.

형태 `del` 딕셔너리[키]

```
sensors = {"모터온도": 78, "펌프압력": 95, "유량": 42}
```

```
del sensors["펌프압력"]  
print(sensors)  
print(len(sensors))
```

```
# del sensors["면적"]
```

```
# KeyError: '면적'
```

```
▶ {'모터온도': 78, '유량': 42}  
▶ 2
```

왜 쓰나 센서를 제거했거나, 잘못 들어온 데이터를 걷어낼 때 씁니다. **키와 값이 한 쌍으로 사라진다는** 점이 중요합니다 — 값만 비우고 키를 남기는 것과는 다릅니다.

응용 del도 없는 키를 지우려 하면 `KeyError`가 납니다. 안전하게 지우려면 `in`으로 먼저 확인하거나, 값만 비우고 싶다면 `None`을 넣는 방법이 있습니다. 둘은 의미가 다르니 목적에 맞게 고릅니다.

```
sensors = {"모터온도": 78, "펌프압력": 95}
```

```
# ① 안전하게 삭제 — 키 자체를 없앤다
```

```
if "면적" in sensors:  
    del sensors["면적"]  
else:  
    print("면적 키 없음 — 삭제 건너뛴")
```

```
# ② 키는 남기고 값만 비운다 — "센서는 있는데 값이 없다"
```

```
sensors["펌프압력"] = None  
print(sensors)
```

```
▶ 면적 키 없음 — 삭제 건너뛴  
▶ {'모터온도': 78, '펌프압력': None}
```

주의

`del`은 **메서드가 아니라 키워드**입니다. `sensors.del("키")`처럼 쓸 수 없고 `del sensors["키"]`가 맞습니다. `del`은 딕셔너리 전용도 아니어서 `del lst[0]`(리스트 요소), `del 변수`(변수 자체)로도 씁니다.

STEP 4

keys · values · items

딕셔너리를 반복문에 넣는 세 가지 방법

딕셔너리를 통째로 `for`에 넣으면 무슨 일이 일어날까요? 여기가 오늘 가장 헛갈리기 쉬운 지점이고, 수업에서도 강사님이 **일부러 오류를 내 보이며** 설명한 부분입니다.

메서드 `.keys()` 키만 모아서 꺼내기

정의 모든 키를 모은 **뷰(view)**를 돌려줍니다. 리스트가 아니라 `dict_keys`라는 전용 타입입니다.

형태 딕셔너리 `.keys()`

```
sensors = {"모터온도": 78, "진동": 0.5, "압력": 95}

print(sensors.keys())
print(type(sensors.keys()))
print(list(sensors.keys()))      # 리스트로 바꿔서 보기
```

```
▶ dict_keys(['모터온도', '진동', '압력'])
▶ <class 'dict_keys'>
▶ ['모터온도', '진동', '압력']
```

왜 쓰나 "등록된 센서가 무엇무엇인가"를 파악할 때 씁니다. 값은 필요 없고 이름 목록만 알고 싶은 상황이죠.

응용 `keys()`로 꺼낸 이름을 **셋으로 바꾸면 어제 배운 집합 연산이 그대로 적용됩니다**. 두 시점의 딕셔너리를 비교해 새로 생긴 센서, 사라진 센서를 찾을 수 있습니다.

```
yesterday = {"모터온도": 77, "진동": 0.4}
today = {"모터온도": 81, "압력": 95}

y_keys = set(yesterday.keys())
t_keys = set(today.keys())

print("신규 센서:", sorted(t_keys - y_keys))
print("사라진 센서:", sorted(y_keys - t_keys))
print("공통 센서:", sorted(t_keys & y_keys))
```

```
▶ 신규 센서: ['압력']
▶ 사라진 센서: ['진동']
▶ 공통 센서: ['모터온도']
```

개념

`for key in sensors:`처럼 딕셔너리를 그냥 넣어도 **키가 나옵니다**. `.keys()`를 생략한 것과 같은 동작입니다. 다만 의도를 드러내려면 `.keys()`를 명시하는 편이 읽기 좋습니다.

메서드 `.values()` 값만 모아서 꺼내기

정의 모든 값을 모은 뷰를 돌려줍니다. 합계·평균처럼 **값만 필요한 계산**에 씁니다.

형태 딕셔너리 `.values()`

```
temps = {"모터온도": 78, "베어링": 92, "펌프": 65}

print(temps.values())
print(sum(temps.values()))
print(sum(temps.values()) / len(temps))
print(max(temps.values()))
```

```
▶ dict_values([78, 92, 65])
▶ 235
▶ 78.33333333333333
▶ 92
```

왜 쓰나 `sum`과 `len`을 조합하면 평균이 한 줄로 나옵니다. 분모가 `len(temps.values())`가 아니라 그냥 `len(temps)`여도 되는 이유는, 딕셔너리의 길이가 곧 값의 개수이기 때문입니다.

응용 `values()`만으로는 **어느 센서의 값인지 알 수 없다**는 한계가 있습니다. 최댓값이 92라는 것은 알아도 그게 베어링인지는 모릅니다. 그래서 "가장 큰 값의 이름"을 찾으려면 다음에 나올 `items()`가 필요합니다.

```
temps = {"모터온도": 78, "베어링": 92, "펌프": 65}

avg = sum(temps.values()) / len(temps)
print(f"평균: {round(avg, 1)}")

over = 0
for v in temps.values():          # 값만 필요하면 values로 충분
    if v > avg:
        over += 1
print(f"평균 초과 센서 수: {over}")
```

```
▶ 평균: 78.3
▶ 평균 초과 센서 수: 1
```

자주 하는 실수

값의 자료형이 섞여 있으면 sum이 실패합니다. {"센서 이름": "motor", "진동": 4.8}처럼 문자열이 끼어 있으면 TypeError가 납니다. 통계를 낼 딕셔너리는 값이 전부 숫자여야 합니다.

메서드 `.items()` 키와 값을 짝으로 꺼내기 — 오늘의 핵심

정의 (키, 값) 튜플을 하나씩 돌려줍니다. 반복문에서 가장 많이 쓰이는 메서드입니다.

형태 `for` 키변수, 값변수 `in` 딕셔너리.`.items()`:

```
sensors = {"모터온도": 78, "진동": 0.5, "압력": 95}
```

```
print(list(sensors.items()))
```

```
for name, value in sensors.items():  
    print(f"{name} | {value}")
```

```
▶ [('모터온도', 78), ('진동', 0.5), ('압력', 95)]  
▶ 모터온도 | 78  
▶ 진동 | 0.5  
▶ 압력 | 95
```

왜 쓰나 이름과 값을 함께 다뤄야 하는 상황이 실무의 대부분입니다. "값이 90을 넘으면 그 센서 이름을 출력하라" 같은 요구는 둘 다 있어야 처리됩니다. 강사님도 "실제로 이쪽을 더 많이 쓴다"고 하셨습니다.

응용 `.items()`가 돌려주는 것이 튜플이라는 점이 중요합니다. 어제 배운 `for name, value in 튜플리스트:` 언패킹이 여기서 그대로 재사용되는 것입니다. 새로운 문법을 배우는 게 아니라 **어제 것이 다시 나온 것**이라고 이해하면 훨씬 쉽습니다.

```
sensors = {"모터온도": 78, "베어링": 92, "펌프": 65}
```

```
# 최댓값과 그 이름을 함께 찾기 — values()로는 불가능
```

```
top_name = ""
```

```
top_value = 0
```

```
for name, value in sensors.items():  
    if value > top_value:  
        top_value = value  
        top_name = name
```

```
print(f"최고 온도: {top_name} ({top_value}°C)")
```

```
▶ 최고 온도: 베어링 (92°C)
```

자주 하는 실수

반복 변수 name, value는 **for문 안에서만** 유효합니다. 반복이 끝난 뒤 쓰려면 위 예시처럼 바깥에 별도 변수를 만들어 담아 두어야 합니다.

items 없이 두 변수로 받으면 왜 안 되나

강사님이 수업 중 일부러 보여주신 오류입니다. 실행해 보면 이렇게 됩니다.

x 오류가 나는 코드

```
sensors = {"모터온도": 78, "진동": 0.5}

for key, value in sensors:          # .items() 를 빠뜨림
    print(key, value)

# ValueError: too many values to unpack (expected 2)
```

원인을 한 줄로 말하면 **딕셔너리를 그냥 순회하면 키 하나씩만 나오기 때문**입니다. "모터온도"라는 문자열 하나를 key, value 두 변수에 나눠 담으려 하니, 글자 4개를 변수 2개에 넣으라는 요구가 되어 실패합니다.

코드	한 번에 나오는 것	받는 변수
for k in sensors:	키 하나 → "모터온도"	1개
for k in sensors.keys():	키 하나 → "모터온도"	1개
for v in sensors.values():	값 하나 → 78	1개
for k, v in sensors.items():	튜플 하나 → ("모터온도", 78)	2개
for k, v in sensors:	키 하나 → "모터온도"	2개 x 불일치

개념

어제 튜플에서 배운 **"변수 개수 = 값 개수"** 규칙이 그대로입니다. `.items()`가 2개짜리 튜플을 주니까 변수도 2개여야 하고, 그냥 순회하면 1개가 나오니까 변수도 1개여야 합니다. 오류 메시지 `too many values to unpack (expected 2)`도 어제 본 것과 똑같은 문장입니다. **딕셔너리를 새로 배우는 게 아니라 튜플 언패킹이 재등장한 것**이라고 보면 됩니다.

세 메서드 비교

메서드	꺼내는 것	받는 변수	언제 쓰나
<code>.keys()</code>	키만	1개	등록된 센서 이름 파악 · 셋으로 비교
<code>.values()</code>	값만	1개	합계 · 평균 · 최댓값 등 통계
<code>.items()</code>	(키, 값) 튜플	2개	이름과 값을 같이 다룸 — 가장 많이 씀

비유

강사님이 든 음식점 주문서 비유가 잘 맞습니다. 주문서가 {"김치찌개": 2, "제육볶음": 3}이라면 — 메뉴 목록만 궁금하면 `keys`, 총 몇 인분인지 궁금하면 `values`의 합, 메뉴별로 몇 인분인지 궁금하면 `items`입니다. 셋 중 마지막이 가장 정보량이 많고, 그래서 가장 자주 쓰입니다.

STEP 5

items 순회와 두 딕셔너리 비교

임계값 판정 패턴

오늘 수업 코드의 결론에 해당하는 부분입니다. **측정값 딕셔너리**와 **임계값 딕셔너리**를 나란히 두고, 같은 키끼리 비교해 경고를 내는 패턴입니다. 실습 5와 8이 모두 이 구조를 요구하고, 앞으로 설비 데이터를 다룰 때 계속 나옵니다.

수업 코드 — 두 딕셔너리

```
# 실제 측정 데이터
values = {"모터 온도": 95, "압력": 88, "진동": 4.5}

# 임계치 데이터 — 같은 키를 공유한다
limits = {"모터 온도": 90, "압력": 90}
```

패턴 두 딕셔너리 대조 같은 키끼리 비교하기

정의 한쪽을 `items()`로 순회하면서, 같은 키를 다른 딕셔너리에서 꺼내 비교합니다.

형태 `for 키, 값 in A.items(): if 값 > B.get(키, 기본값):`

```
values = {"모터 온도": 95, "압력": 88, "진동": 4.5}
limits = {"모터 온도": 90, "압력": 90}

for name, value in values.items():
    print(f"{name} | {value}")

    if value > limits.get(name, 0):
        print("경고")
```

```
▶ 모터 온도 | 95
▶ 경고
▶ 압력 | 88
▶ 진동 | 4.5
▶ 경고 ← 이게 문제입니다
```

왜 쓰나 센서마다 기준이 다르므로 임계값을 코드에 박을 수 없습니다. **기준도 데이터로 관리**하고 같은 키로 짝지어 비교하는 것이 정석입니다. 기준이 바뀌면 `limits` 딕셔너리만 고치면 됩니다.

응용 `limits`에 키가 없을 때 `limits[name]`을 쓰면 `KeyError`로 멈춥니다. 그래서 `get`을 쓰는데, **기본값을 무엇으로 두느냐**가 결과를 완전히 바꿉니다. 위 출력에서 진동 4.5가 경고로 뜬 이유가 바로 그것입니다 — 자세한 분석은 STEP 8에서 다룹니다.

자주 하는 실수

이 패턴은 **두 딕셔너리가 같은 키 이름을 쓴다는 전제** 위에서만 동작합니다. 한쪽이 "모터 온도", 다른 쪽이 "모터온도"(공백 없음)면 서로 다른 키라 영원히 매칭되지 않습니다. 오류도 안 납니다.

패턴 조건에 맞는 키 모으기 `items + 조건문 + append`

정의 `items()` 순회 중 조건을 만족하는 키를 빈 리스트(또는 셋)에 모으는 패턴.

```
values = {"모터 온도": 95, "압력": 88, "진동": 4.5, "유량": 42}
limits = {"모터 온도": 90, "압력": 90, "진동": 5.0, "유량": 50}

warning = []
for name, value in values.items():
    if value > limits[name]:
        warning.append(name)

print("경고 센서:", warning)
print("정상 센서 수:", len(values) - len(warning))
```

▶ 경고 센서: ['모터 온도']
▶ 정상 센서 수: 3

왜 쓰나 8일차에 배운 "**빈 리스트 → for → if → append**" 네 줄 패턴이 딕셔너리에도 그대로 적용됩니다. 순회 대상이 리스트에서 `딕셔너리.items()`로 바뀌었을 뿐입니다.

응용 모으는 그릇을 **리스트 대신 셋으로 바꾸면 중복이 자동으로 제거**됩니다. 한 설비에서 여러 항목이 동시에 초과할 수 있는 상황에서는 셋이 훨씬 안전합니다. 실습 5에서 실제로 이 문제가 발생하는데, STEP 10에서 다룹니다.

```
warning = set()                                # 리스트 대신 셋

readings = [
    {"이름": "bearing", "진동": 8.2, "속도": 1750},
    {"이름": "pump", "압력": 12.4, "점도": 46.2},
]

limits = {"진동": 5.0, "속도": 1500, "압력": 10.0, "점도": 45.0}

for r in readings:
    name = r["이름"]
    for key, value in r.items():
        if key == "이름":
            continue
        if key in limits and value > limits[key]:
            warning.add(name)    # 두 번 넘어도 한 번만 담긴다

print("점검 대상:", sorted(warning))
```

▶ 점검 대상: ['bearing', 'pump']

개념

continue는 8일차에 배운 키워드입니다. "이번 회차는 건너뛰고 다음으로" — 여기서는 "이름" 키가 측정값이 아니므로 비교에서 제외하는 데 씁니다.

센서 이름을 키에 섞을 때의 함정

오늘 코드에서 반복해 나온 구조가 하나 있습니다. 딕셔너리 안에 **이름(문자열)**과 **측정값(숫자)**이 섞여 있는 형태입니다.

섞여 있는 구조

```
{"센서 이름": "motor_vibration", "진동": 4.8, "온도": 78.5}
```

하려는 일	이 구조에서 생기는 문제	대응
<code>sum(d.values())</code>	<code>TypeError</code> — 문자열과 숫자를 더할 수 없다	이름 키를 빼고 합산
<code>for k, v in d.items()</code>	이름 항목까지 측정값처럼 비교됨	<code>if k == "센서 이름": continue</code>
평균 내기	분모에 이름 항목까지 포함돼 값이 틀어짐	측정 항목만 세어 나눈다

개념

더 깔끔한 대안은 **이름을 값이 아니라 키로 올리는 것**입니다. {"motor_vibration": {"진동": 4.8, "온도": 78.5}} 형태로 만들면 바깥 키가 곧 센서 이름이 되고, 안쪽 딕셔너리는 순수하게 측정값만 갖습니다. 실습 6의 중첩 딕셔너리가 정확히 이 구조이고, 실무에서도 이쪽이 더 흔합니다. STEP 7에서 다시 다룹니다.

구조를 바꾼 코드

이름을 키로 올린 구조 — 섞임 문제가 사라진다

```
equipment = {  
    "motor_vibration": {"진동": 4.8, "온도": 78.5},  
    "pump_pressure": {"압력": 2.1, "유량": 45.2},  
}  
  
for name, data in equipment.items():  
    total = sum(data.values())          # 이제 전부 숫자라 안전  
    print(f"{name}: 항목 {len(data)}개, 합 {round(total, 1)}")
```

▶ motor_vibration: 항목 2개, 합 83.3

▶ pump_pressure: 항목 2개, 합 47.3

STEP 6

update 와 zip

편하지만 사고가 나는 두 기능

강사님이 이 두 기능을 소개하면서 "저는 개인적으로 잘 쓰지 않는다"고 여러 번 말씀하셨습니다. 기능이 나빠서가 아니라 **사람이 실수하기 좋은 구조**라서입니다. 그래서 이 STEP은 "어떻게 쓰는가"만큼 "왜 조심해야 하는가"에 무게를 둡니다.

메서드 `.update()` 여러 쌍을 한 번에 반영

정의 다른 딕셔너리의 키-값을 통째로 넣습니다. **있는 키는 덮어쓰고, 없는 키는 추가합니다.**

형태 `딕셔너리.update(다른_딕셔너리)`

```
sensors = {"센서 이름": "generator_volt", "전압": 220.4, "전류": 15.1}
new_data = {"센서 이름": "hydraulic_oil", "압력": 12.4, "점도": 46.2}

print(sensors.update(new_data))    # 반환값은 None
print(sensors)
```

- ▶ None
- ▶ {'센서 이름': 'hydraulic_oil', '전압': 220.4, '전류': 15.1, '압력': 12.4, '점도': 46.2}

왜 쓰나 키를 하나씩 `d[k] = v`로 넣으면 줄이 길어집니다. 한 번에 여러 값을 갱신해야 할 때 `update` 한 줄이면 끝납니다.

응용 출력을 자세히 보면 문제가 보입니다. "센서 이름"이 `generator_volt`에서 `hydraulic_oil`로 **조용히 덮어써졌는데**, 전압·전류는 `generator_volt`의 값 그대로 남아 있습니다. 결과적으로 **이름과 데이터가 어긋난 딕셔너리**가 만들어졌습니다. 오류는 나지 않습니다.

```
# 덮어쓰기 전에 무엇이 겹치는지 확인하는 습관
sensors = {"센서 이름": "generator_volt", "전압": 220.4}
new_data = {"센서 이름": "hydraulic_oil", "압력": 12.4}

overlap = set(sensors.keys()) & set(new_data.keys())
if overlap:
    print("덮어쓰게 될 키:", sorted(overlap))
    for k in sorted(overlap):
        print(f"  {k}: {sensors[k]} → {new_data[k]}")

sensors.update(new_data)
```

▶ 덮어쓰게 될 키: ['센서 이름']
 ▶ 센서 이름: generator_volt → hydraulic_oil

자주 하는 실수

강사님의 경고 그대로입니다 — **"어떤 키가 덮어써질지 손으로 따지다 보면 하루 종일 갈 수도 있다. 가급적 피할 수 있으면 피하는 것도 좋다."** 두 딕셔너리가 같은 키 집합을 갖는다는 확신이 있을 때만 쓰는 것이 안전합니다.

.update()도 .sort()·.add()처럼 **None**을 반환하므로 `d = d.update(x)`로 쓰면 딕셔너리가 사라집니다.

내장함수 `zip()` 두 리스트를 짝지어 묶기

정의 두 리스트를 앞에서부터 하나씩 짝지어 줍니다. `dict()`로 감싸면 딕셔너리가 됩니다.

형태 `dict(zip(키_리스트, 값_리스트))`

```
names = ["모터온도", "진동", "압력"]
values = [78, 0.5, 95]

sensors = dict(zip(names, values))
print(sensors)
```

▶ {'모터온도': 78, '진동': 0.5, '압력': 95}

왜 쓰나 파일이나 엑셀에서 데이터를 읽으면 **헤더 행과 데이터 행이 따로** 오는 경우가 많습니다. 첫 줄은 컬럼 이름, 둘째 줄은 값 — 이 둘을 붙여 딕셔너리로 만드는 것이 `zip`의 대표 용도입니다.

응용 이름이 `zip`인 이유는 **지퍼**입니다. 강사님이 압축 파일(.zip)과 연결해 설명하셨는데, 공통점은 "여러 개를 하나로 묶는다"입니다. 지퍼의 양쪽 이가 앞에서부터 하나씩 맞물리는 그림을 떠올리면 동작이 정확히 그려집니다.

```
# 헤더 + 데이터 행 → 딕셔너리 리스트
header = ["설비", "온도", "진동"]
rows = [
    ["컨베이어_01", 78, 2.1],
    ["용접기_02", 92, 5.4],
]

records = []
for row in rows:
    records.append(dict(zip(header, row)))

for r in records:
    print(r)
```

- ▶ {'설비': '컨베이어_01', '온도': 78, '진동': 2.1}
- ▶ {'설비': '용접기_02', '온도': 92, '진동': 5.4}

자주 하는 실수

개수가 다르면 짧은 쪽에 맞춰 조용히 잘립니다. 오류가 나지 않는 것이 가장 위험한 지점입니다.

zip의 개수 불일치 — 강사님이 일부러 보여준 실패 사례

수업에서 잘 되는 코드만 보여주면 실습에서 당황한다며, 일부러 틀린 케이스를 만들어 보여주신 부분입니다.

```
x 개수가 어긋난 경우
names = ["모터온도", "진동", "압력", "유량"]      # 키 4개
values = [78, 0.5, 95]                          # 값 3개

sensors = dict(zip(names, values))
print(sensors)
print("만들어진 쌍:", len(sensors), "개")
```

- ▶ {'모터온도': 78, '진동': 0.5, '압력': 95}
- ▶ 만들어진 쌍: 3 개
- ▶
- ▶ ↑ "유량"이 오류 없이 사라졌습니다

자주 하는 실수

유량이 그냥 없어집니다. 프로그램은 정상 종료되고, 결과 딕셔너리도 멀쩡해 보입니다. 나중에 `sensors["유량"]`을 조회하다 `KeyError`가 나서야 문제를 알게 되는데, 그때는 원인이 `zip`이라는 걸 찾기가 어렵습니다. 반대로 값이 더 많은 경우도 마찬가지로 남은 값이 버려집니다. **zip을 쓸 때는 `len(names) == len(values)`를 먼저 확인하는 것이 유일한 방어책입니다.**

✓ 확인 후 사용

```
names = ["모터온도", "진동", "압력", "유량"]
values = [78, 0.5, 95]

if len(names) != len(values):
    print(f"⚠ 개수 불일치 — 키 {len(names)}개 / 값 {len(values)}개")
    print("   누락될 키:", names[len(values):])
else:
    sensors = dict(zip(names, values))
    print(sensors)
```

▶ ⚠ 개수 불일치 — 키 4개 / 값 3개

▶ 누락될 키: ['유량']

주의

강사님이 남긴 정리를 그대로 옮기면 — `items`는 앞으로 계속 쓰게 될 코드이고, `zip`과 `update`는 "그런 게 있었지" 정도로 알아두었다가 필요할 때 찾아 쓰면 되는 기능입니다. 셋 다 외울 필요는 없고, 어떤 상황에서 위험한지만 기억해 두면 충분합니다.

STEP 7

딕셔너리 리스트와 중첩 딕셔너리

실무에서 가장 많이 보는 구조

강사님이 오늘 여러 번 강조한 말이 있습니다 — "딕셔너리 하나만 쓰이는 사례가 거의 없고, 리스트 하나만 쓰이는 사례도 거의 없다." 실제 데이터는 둘이 겹쳐진 형태로 옵니다. 여기가 오늘의 도착점입니다.

패턴 `[{...}, {...}, ...]` 딕셔너리들을 담은 리스트

정의 리스트의 각 요소가 딕셔너리인 구조. **엑셀 표 한 장**과 정확히 대응합니다.

```
sensors_list = [
    {"센서 이름": "motor_vibration", "진동": 4.8, "온도": 78.5},
    {"센서 이름": "pump_pressure", "압력": 2.1, "유량": 45.2},
    {"센서 이름": "compressor_temp", "온도": 92.4, "진동": 1.8},
]

print(type(sensors_list))
print(sensors_list[0])          # 첫 번째 행 전체
print(sensors_list[0]["센서 이름"]) # 그 행의 특정 칸
```

```
▶ <class 'list'>
▶ {'센서 이름': 'motor_vibration', '진동': 4.8, '온도': 78.5}
▶ motor_vibration
```

왜 쓰나 강사님의 엑셀 비유가 정확합니다 — **첫 줄이 컬럼 이름, 그 아래 각 줄이 한 건의 데이터**. 한 줄(행)을 딕셔너리 하나로 만들고, 그 줄들을 리스트에 모으면 표가 그대로 코드가 됩니다. 센서 데이터는 시간 단위로 계속 쌓이므로 정확히 이 모양이 됩니다.

응용 어제 배운 **튜플 리스트**와 비교하면 차이가 분명합니다. 튜플 리스트는 `[ (이름, 값) ]`처럼 자리로 구분해 항목 수가 고정이어야 하지만, 딕셔너리 리스트는 **행마다 키가 달라도 됩니다**. 위 데이터에서 첫 행은 진동·온도, 둘째 행은 압력·유량으로 서로 다른데도 문제가 없습니다.

```
sensors_list = [
    {"센서 이름": "motor_vibration", "진동": 4.8, "온도": 78.5},
    {"센서 이름": "pump_pressure", "압력": 2.1, "유량": 45.2},
]

# 행 단위 수정 — 리스트 인덱스 + 딕셔너리 키
sensors_list[0]["센서 이름"] = "motor_vib"
print(sensors_list[0])

# 전체 순회
for row in sensors_list:
    name = row["센서 이름"]
    items = len(row) - 1          # 이름 칸을 빼면 측정 항목 수
    print(f"{name}: 측정 항목 {items}개")
```

- ▶ {'센서 이름': 'motor_vib', '진동': 4.8, '온도': 78.5}
- ▶ motor_vib: 측정 항목 2개
- ▶ pump_pressure: 측정 항목 2개

설비 적용

앞으로 pandas를 배우면 `pd.DataFrame(sensors_list)` 한 줄로 이 리스트가 표가 됩니다. 지금 손으로 만드는 구조가 그대로 입력이 되는 것이니, 이 형태에 익숙해지는 것이 곧 다음 단계 준비입니다.

패턴 {키: {키: 값}} 중첩 딕셔너리

정의 딕셔너리의 값으로 또 딕셔너리를 넣은 구조. **설비 이름이 키가 되고 그 아래 상세 정보가 딸립니다.**

```
factory = {
    "Compressor_A": {"status": "정상", "temperature": 42.5, "pressure": 3.2},
    "Pump_B": {"status": "경고", "temperature": 68.1, "pressure": 1.8},
    "Motor_C": {"status": "정상", "temperature": 35.4, "vibration": 2.5},
}

print(factory["Pump_B"])          # 설비 전체 정보
print(factory["Pump_B"]["temperature"]) # 그 설비의 특정 값
```

- ▶ {'status': '경고', 'temperature': 68.1, 'pressure': 1.8}
- ▶ 68.1

왜 쓰나 딕셔너리 리스트와 무엇이 다른가 — **찾는 방법이 다릅니다.** 리스트에 담으면 특정 설비를 찾으려고 전부 순회해야 하지만, 중첩 딕셔너리는 `factory["Pump_B"]` 한 번에 도달합니다. **설비 이름으로 바로 조회할 일이 많으면** 이쪽이 유리합니다.

응용 바깥을 `items()`로 순회하면 설비명과 상세 정보를 동시에 받습니다. 안쪽 딕셔너리는 다시 `[키]`나 `get`으로 파고듭니다. 2단으로 들어가는 것이지 새로운 문법이 아닙니다.

```
for name, info in factory.items():
    if info["status"] == "경고":
        print(f"△ {name}: {info['temperature']}°C - 점검 필요")
```

```
# 안쪽에 없을 수도 있는 키는 get으로
for name, info in factory.items():
    vib = info.get("vibration", -1)
    mark = "미측정" if vib < 0 else f"{vib}"
    print(f"{name} 진동: {mark}")
```

- ▶ △ Pump_B: 68.1°C - 점검 필요
- ▶ Compressor_A 진동: 미측정
- ▶ Pump_B 진동: 미측정
- ▶ Motor_C 진동: 2.5

개념

`factory["Pump_B"]["temperature"]`처럼 대괄호가 이어지는 것은 어제 중첩 튜플에서 `t[2][0]`을 쓴 것과 같은 원리입니다. **바깥부터 안쪽으로 한 겹씩** 들어간다고 읽으면 됩니다.

문법 값에 리스트 담기 한 키에 여러 값 매달기

정의 딕셔너리 값 자리에 리스트나 튜플을 넣어 **한 이름표에 여러 값**을 저장합니다.

```
groups = {
    "BTS": ["RM", "진", "슈가", "제이홉"],
    "블랙핑크": ["지수", "제니", "로제", "리사"],
}

print(groups["BTS"])
print(groups["BTS"][0])          # 리스트의 첫 요소
print(len(groups["블랙핑크"]))   # 멤버 수

▶ ['RM', '진', '슈가', '제이홉']
▶ RM
▶ 4
```


왜 쓰나 값이 하나로 정해지지 않는 경우가 있습니다. 수업에서 든 예가 "부반장이 두 명이면?"이었는데, 값을 리스트로 바꾸면 몇 명이든 답을 수 있습니다. 센서 데이터에서는 **한 센서의 시간별 측정 이력**이 여기 해당합니다.

응용 모터온도를 1분마다 재면 한 시간에 60개입니다. 이 60개를 키 60개로 만들 수는 없으니, "모터온도": [78, 79, 81, ...]처럼 리스트로 매답니다. 그러면 그 리스트에 어제 배운 `sum·max·sorted`를 그대로 적용할 수 있습니다.

```
history = {
    "모터온도": [78, 79, 81, 85, 92],
    "진동": [0.4, 0.5, 0.5, 0.6, 0.5],
}

for name, values in history.items():
    avg = sum(values) / len(values)
    print(f"{name}: 최근 {len(values)}회 · 평균 {round(avg, 1)} · 최대 {max(values)}")

print("모터온도 마지막 값:", history["모터온도"][-1])
```

- ▶ 모터온도: 최근 5회 · 평균 83.0 · 최대 92
- ▶ 진동: 최근 5회 · 평균 0.5 · 최대 0.6
- ▶ 모터온도 마지막 값: 92

주의

값에는 무엇이든 넣을 수 있습니다 — 숫자·문자열·리스트·튜플·셋·또 다른 딕셔너리. **제약이 있는 쪽은 키뿐**입니다(STEP 1). 값이 자유롭기 때문에 실무 데이터의 어떤 형태든 딕셔너리로 표현할 수 있습니다.

두 구조, 어느 쪽을 고를 것인가

	딕셔너리 리스트	중첩 딕셔너리
모양	[{...}, {...}]	{"A": {...}, "B": {...}}
특정 설비 찾기	전부 순회해야 함	키로 바로 조회
순서	있음 — 시간 순서 보존	넣은 순서
같은 설비 여러 건	가능 — 시계열 기록에 적합	불가 — 키가 유일해야 함
어울리는 데이터	시간별로 쌓이는 측정 로그	설비 마스터 정보 · 현재 상태

설비 적용

실무에서는 둘 다 씁니다. 설비 제원처럼 고정된 정보는 중첩 딕셔너리로 두고 이름으로 바로 찾고, 측정 로그는 딕셔너리 리스트로 쌓아 시간 순서를 보존합니다. 강사님이 "하락적으로 봐야 한다" — 즉 계층적으로 봐야 한다고 하신 것이 이 얘기입니다. **딕셔너리 안에 리스트, 리스트 안에 딕셔너리가 몇 겹으로 겹친 구조까지 상상이 되면 시니어에 가까워지는 것**이라고 하셨습니다. 웹 서비스에서 서버와 주고받는 데이터도 대부분 이 형태(JSON)입니다.

두 구조 사이를 오가기

실무에서는 같은 데이터를 **상황에 따라 다른 구조로 바꿔** 씁니다. 시간순으로 쌓을 땐 딕셔너리 리스트, 이름으로 찾을 땐 중첩 딕셔너리 — 둘 사이 변환은 반복문 몇 줄이면 됩니다.

구조 변환

```
records = [
    {"센서 이름": "motor_vibration", "진동": 4.8, "온도": 78.5},
    {"센서 이름": "pump_pressure", "압력": 2.1, "유량": 45.2},
]

# 딕셔너리 리스트 → 중첩 딕셔너리
equipment = {}
for row in records:
    name = row["센서 이름"]
    info = {}
    for key, value in row.items():
        if key == "센서 이름":
            continue
        info[key] = value
    equipment[name] = info

print(equipment)
print("바로 조회:", equipment["pump_pressure"]["압력"])

▶ {'motor_vibration': {'진동': 4.8, '온도': 78.5}, 'pump_pressure': {'압력': 2.1, '유량': 45.2}}
▶ 바로 조회: 2.1
```

변환하고 나면 `equipment["pump_pressure"]`로 한 번에 도달합니다. **리스트였을 때는 전부 순회하며 이름을 비교해야 했던 일이 사라집니다.** 어제 배운 셋의 `in`이 리스트보다 빨랐던 것과 같은 이유입니다 — **이름으로 바로 찾는 구조는 뒤지지 않습니다.**

설비 적용

센서가 10개일 땐 어느 쪽이든 상관없습니다. 하지만 설비 수천 대의 상태를 조회하는 화면이라면 얘기가 달라집니다. 리스트를 매번 처음부터 훑으면 화면이 느려지고, 딕셔너리로 두면 즉시 나옵니다. **자료구조 선택이 곧 응답 속도**라는 감각이 여기서도 똑같이 적용됩니다.

이 구조에는 이름이 있다 — JSON

강사님이 **"로그인할 때 서버와 주고받는 데이터가 거의 이런 형태"**라고 하신 것이 JSON입니다. 딕셔너리와 리스트가 겹겹이 쌓인 구조에 붙은 이름이고, 오늘날 거의 모든 웹 서비스와 API가 이 형식으로 데이터를 주고받습니다.

설비 API가 보내주는 데이터 (JSON)

```
{
  "line": "A동_1라인",
  "timestamp": "2026-08-03T14:30:00",
  "sensors": [
    {"name": "motor_vibration", "value": 4.8, "unit": "mm/s", "status": "정상"},
    {"name": "compressor_temp", "value": 92.4, "unit": "C", "status": "경고"}
  ],
  "summary": {"total": 2, "warning": 1}
}
```

겉모습이 파이썬 딕셔너리와 거의 같습니다. 실제로 파이썬은 JSON을 읽으면 **딕셔너리와 리스트로 바뀌서** 건네줍니다. 그래서 오늘 배운 접근법이 그대로 통합니다.

꺼내려는 것	코드	결과
라인 이름	<code>data["line"]</code>	"A동_1라인"
센서 목록	<code>data["sensors"]</code>	딕셔너리 2개가 든 리스트
첫 센서 이름	<code>data["sensors"][0]["name"]</code>	"motor_vibration"
경고 건수	<code>data["summary"]["warning"]</code>	1
경고 센서만	<code>for s in data["sensors"]: if s["status"]=="경고"</code>	compressor_temp

연결

아직 JSON을 다루는 문법(json 모듈)은 배우지 않았지만, **구조를 읽는 능력은 오늘 이미 갖췄습니다**. 대괄호로 한 겹씩 들어가고, 리스트면 순회하고, 딕셔너리면 `items()`로 푸는 것 — 그게 전부입니다. 나중에 설비 API에서 데이터를 받아오는 코드를 짤 때 새로 배울 것은 **"받아오는 방법"뿐이고, 받은 뒤에 다루는 방법은 오늘 것 그대로**입니다.

STEP 8

내 코드 점검 — 16_dictionary.py

기본값 0이 만든 오경보

오늘 수업 코드를 그대로 실행해 보면 마지막 부분에서 이상한 결과가 나옵니다. 오류는 없고 결과만 틀린 유형이라 눈으로만 봐서는 지나치기 쉽습니다.

실행 결과

```
values = {"모터 온도": 95, "압력": 88, "진동": 4.5}
limits = {"모터 온도": 90, "압력": 90}
```

```
for name, value in values.items():
    print(f"{name} | {value}")

    if value > limits.get(name, 0):
        print("경고")
```

```
▶ 모터 온도 | 95
▶ 경고
▶ 압력 | 88
▶ 진동 | 4.5
▶ 경고 ← 진동에 임계값이 없는데 경고가 뜬다
```

무슨 일이 일어났나

센서	측정값	limits.get(name, 0)	비교	결과
모터 온도	95	90 (있음)	95 > 90	경고 ✓ 맞음
압력	88	90 (있음)	88 > 90	통과 ✓ 맞음
진동	4.5	0 (없어서 기본값)	4.5 > 0	경고 ✗ 오경보

limits에 "진동" 키가 없으니 get이 기본값 0을 돌려주고, 측정값 4.5는 당연히 0보다 큼니다. 임계값이 정의되지 않은 센서가 무조건 경고로 잡히는 구조입니다. 센서가 100개인데 임계값이 3개만 정의돼 있으면 97개가 전부 경고로 뜹니다.

자주 하는 실수

이것을 현장 용어로 **오경보(false alarm)**라고 합니다. 예지보전 시스템에서 가장 경계하는 문제인데, 경보가 자주 틀리면 작업자가 경보 자체를 무시하게 되어 정작 진짜 이상을 놓치기 때문입니다. **기술적으로는 한 글자 차이지만 현장에서는 시스템 신뢰도의 문제입니다.**

고치는 방법 세 가지

상황에 따라 답이 다릅니다. **"임계값이 없는 센서를 어떻게 취급할 것인가"**를 먼저 정해야 합니다.

① 기본값을 무한대로

```
# ① 임계값 없으면 경고하지 않는다 — 가장 안전 (권장)
#     무한대를 기본값으로 두면 어떤 값도 이를 넘지 못한다
for name, value in values.items():
    if value > limits.get(name, float("inf")):
        print(f"{name} 경고")
```

▶ 모터 온도 경고

② in으로 먼저 거른다

```
# ② 임계값이 있는 센서만 비교한다 — 의도가 가장 명확
for name, value in values.items():
    if name not in limits:
        print(f"{name}: 임계값 미설정 — 판정 제외")
        continue
    if value > limits[name]:
        print(f"{name} 경고")
```

▶ 모터 온도 경고

▶ 진동: 임계값 미설정 — 판정 제외

③ 데이터를 채운다

```
# ③ 임계값 딕셔너리를 완성한다 — 근본 해결
limits = {"모터 온도": 90, "압력": 90, "진동": 5.0}

for name, value in values.items():
    if value > limits[name]:                # 이제 get 없이 대괄호로 안전
        print(f"{name} 경고")
```

▶ 모터 온도 경고

개념

`float("inf")`는 **양의 무한대**를 뜻하는 파이썬 값입니다. 어떤 숫자와 비교해도 항상 더 크므로, "이 조건은 절대 만족하지 않는다"를 표현할 때 씁니다. 최솟값을 찾을 때 초기값으로 쓰기도 합니다 — 어제 종합 실습에서 `min_val = 0`으로 시작해 문제가 됐던 자리에 `float("inf")`를 넣으면 첫 비교부터 정상 동작합니다.

기본값을 정할 때의 판단 기준

상황	기본값	이유
합계에 더할 값	0	없는 값은 0으로 더해도 합이 안 변한다
측정값 결측 표시	-1	음수는 실제 측정값이 될 수 없어 구분된다
임계값 (초과 판정)	<code>float("inf")</code>	아무도 넘지 못하게 해 오경보를 막는다
최솟값 찾기 초기값	<code>float("inf")</code>	첫 비교부터 갱신되게 만든다
값이 없음을 알아야 함	None	기본값 없이 <code>get</code> 을 쓰면 자동으로 None

주의

강사님이 수업에서 "**0도 원래 값이 있다고 판단할 수 있기 때문에 마이너스를 쓰는 사례도 있다**"고 하신 것이 정확히 이 얘기입니다. 그리고 실습 1을 시연하실 때 실제로 기본값을 -1로 두고 보여주셨습니다. **기본값은 습관적으로 0을 쓰는 게 아니라 매번 의도를 갖고 고르는 것입니다.**

그 밖에 남은 부분

위치	상태	할 일
# 필요없는 key와 그 value를 삭제	빈 채로 남음	<code>del sensors_list[0]["온도"]</code>
now_weight 주석의 의문	실행 결과 460 — 정상	<code>get("하중")</code> 은 450만 가져온다
limits에 진동 키 없음	오경보 발생	위 세 방법 중 택일

개념

코드에 남기신 주석 — # 여기서 속도라는 1.2가 `motor_degree`에 나오는가? — 에 답하면, **나오지 않습니다.** `s_dict.get("하중")`은 "하중" 키에 찍지어진 값 하나(450)만 가져옵니다. 딕셔너리는 키와 값이 **1:1로 묶여** 있어서 옆 항목이 떨어 오는 일이 없습니다. 리스트라면 인덱스를 하나 잘못 세서 옆 값을 가져올 수 있지만, 딕셔너리에서는 그 실수 자체가 불가능합니다. **이름으로 찾는다는 것의 가장 큰 장점**이 이것입니다.

STEP 9

실습 1~4 복기

제출 코드 점검

교안 실습은 총 8개인데, **강사님이 직접 시연한 것은 1번뿐**이고 2~8번은 각자 풀어 제출하는 과제였습니다. 여기서는 제출하신 06_02_example.py를 요구사항과 하나씩 대조합니다.

워밍업 — 포켓몬 진화 딕셔너리

실습에 들어가기 전 직접 만드신 코드입니다. **딕셔너리 리스트 + 이중 반복 + items 순회**를 한 번에 담고 있어서, 오늘 배운 구조를 정확히 이해했다는 것을 보여줍니다.

제출 코드 — 워밍업

```
pkm_dict = [
    {"진화 1단계": "이상해씨", "진화 2단계": "이상해풀", "진화 3단계": "이상해꽃"},
    {"진화 1단계": "파이리", "진화 2단계": "리자드", "진화 3단계": "리자몽"},
    # ...
]
```

```
for d in pkm_dict:          # 바깥 — 리스트 순회 (딕셔너리 하나씩)
    print(d)
    for key, val in d.items(): # 안쪽 — 딕셔너리 순회 (키·값 짝으로)
        print(f"{key} | {val}")
```

- ▶ {'진화 1단계': '이상해씨', '진화 2단계': '이상해풀', '진화 3단계': '이상해꽃'}
- ▶ 진화 1단계 | 이상해씨
- ▶ 진화 2단계 | 이상해풀
- ▶ 진화 3단계 | 이상해꽃
- ▶ ...

개념

이중 반복문에서 **바깥은 그냥 순회**(for d in pkm_dict), **안쪽은 .items() 순회로 나눈 것이 정확합니다**. **바깥 대상은 리스트라 요소가 하나씩(딕셔너리) 나오고, 안쪽 대상은 딕셔너리라 키·값 두 개를 받으려면 .items()가 필요합니다**. 같은 for인데 대상 자료형에 따라 쓰는 법이 달라진다는 것을 코드로 구분해 낸 셈입니다.

실습 1 — 딕셔너리 만들고 다루기

요구사항 세 개(생성 · 조회 · 추가 · 수정 · get과 in)를 모두 충족했습니다. 특히 `get("유량", 0)`으로 기본값을 준 부분과 `in`으로 있는 키 · 없는 키를 각각 확인한 부분이 정확합니다.

제출 코드 — 실습 1

```
sensors_dict = {"센서명": "compressor_temp", "온도": 92.4, "진동": 1.8}
print("키 조회 결과:", sensors_dict.get("온도"))
sensors_dict["압력"] = 2.1 # 새 키 추가
print("get으로 없는 키 조회(유량):", sensors_dict.get("유량", 0))
print("in으로 키 존재 확인(진동):", "진동" in sensors_dict)
print("in으로 키 존재 확인(회전수):", "회전수" in sensors_dict)
```

- ▶ 키 조회 결과: 92.4
- ▶ 압력 추가 결과 - 압력값: 2.1
- ▶ get으로 없는 키 조회(유량): 0
- ▶ in으로 키 존재 확인(진동): True
- ▶ in으로 키 존재 확인(회전수): False

주의

요구사항 ②는 "키로 값을 꺼내고 새 키로 추가 / 기존 키로 수정" 세 가지였는데, 추가는 했지만 기존 키로 수정하는 줄이 빠져 있습니다. `sensors_dict["온도"] = 95.0` 한 줄을 넣고 전후를 출력하면 "같은 문법이 추가와 수정을 모두 처리한다"는 오늘의 핵심이 눈에 보입니다. 그리고 기본값을 0으로 둔 것은 실습 요구사항 그대로지만, STEP 8에서 본 것처럼 **결측과 실제 0을 구분해야 하는 상황에서는 -1이 더 안전합니다.**

실습 2 — update로 여러 값 한 번에 갱신하기

①과 ②는 정확한데 ③이 통째로 빠졌습니다. 요구사항은 `del`로 특정 키를 삭제하고 `len`으로 개수를 확인하는 것이었습니다.

제출 코드 — 실습 2

```
sensor_dict = {"센서 이름": "generator_volt", "전압": 220.4, "전류": 15.1}
new_data = {"센서 이름": "hydraulic_oil", "압력": 12.4, "점도": 46.2}
print(f"센서 딕셔너리: {sensor_dict}")
sensor_dict.update(new_data)
print(f"new_data update: {sensor_dict}")
```

③ del + len 이 없음

▶ 센서 딕셔너리: {'센서 이름': 'generator_volt', '전압': 220.4, '전류': 15.1}
▶ new_data update: {'센서 이름': 'hydraulic_oil', '전압': 220.4, '전류': 15.1, '압력': 12.4, '점도': 46.2}

완성본

```
sensor_dict = {"센서 이름": "generator_volt", "전압": 220.4, "전류": 15.1}
new_data = {"센서 이름": "hydraulic_oil", "압력": 12.4, "점도": 46.2}
```

```
print("원본:", sensor_dict)
sensor_dict.update(new_data)
print("update 후:", sensor_dict)
```

```
del sensor_dict["전류"]          # ③ 특정 키 삭제
print("del 후:", sensor_dict)
print("센서 수:", len(sensor_dict))
```

▶ 원본: {'센서 이름': 'generator_volt', '전압': 220.4, '전류': 15.1}
▶ update 후: {'센서 이름': 'hydraulic_oil', '전압': 220.4, '전류': 15.1, '압력': 12.4, '점도': 46.2}
▶ del 후: {'센서 이름': 'hydraulic_oil', '전압': 220.4, '압력': 12.4, '점도': 46.2}
▶ 센서 수: 4

개념

출력을 보면 STEP 6에서 다룬 문제가 그대로 보입니다. "센서 이름"은 hydraulic_oil로 바뀌었는데 전압·전류는 generator_volt의 값이 그대로 남아 있습니다. **실습 데이터라 그렇지만, 실제로 이런 딕셔너리가 만들어지면 이름과 데이터가 어긋난 상태가 됩니다.** update가 왜 조심스러운 기능인지 이 출력 한 줄이 잘 보여줍니다.

실습 3 — 딕셔너리로 통계 내기

로직은 정확합니다. `items()` 순회로 합계와 최댓값을 한 번에 구한 구조가 좋습니다. 다만 초기값 두 개에 손볼 곳이 있습니다.

제출 코드 — 실습 3

```
total = 0
top_val = 0
top_val_name = 0          # ← 이름이 들어갈 변수를 숫자 0으로 초기화

for name, value in sensor_data.items():
    total += value
    if top_val < value:
        top_val = value
        top_val_name = name

print(f"센서값 평균: {round(total / len(sensor_data), 1)}")
print(f"가장 큰 값({top_val_name}): {top_val}")
```

▶ 센서값 평균: 38.2

▶ 가장 큰 값(VIB_SENSOR_05): 92.4

변수	현재	문제	권장
<code>top_val_name = 0</code>	정수 0	문자열이 들어갈 자리에 숫자 — 데이터가 비면 0이 출력됨	<code>= ""</code>
<code>top_val = 0</code>	정수 0	모든 값이 음수면 최댓값을 못 찾음	<code>= float("-inf")</code>

개념

어제 종합 실습에서 `min_val = 0` 때문에 최솟값이 항상 0으로 나온 것과 같은 계열의 문제입니다. 이번에는 데이터가 전부 양수라 결과가 맞게 나왔을 뿐입니다. 초기값은 "아직 아무것도 안 봤을 때의 값"이므로, 최댓값을 찾을 땐 가능한 가장 작은 값, 최솟값을 찾을 땐 가장 큰 값으로 두는 것이 논리적으로 맞습니다. `float("-inf")` / `float("inf")`가 그 역할입니다. 더 간단하게는 첫 번째 값으로 초기화해도 됩니다.

실습 4 — zip으로 센서명-값 매핑하기

①②는 정확한데 ③(`items`로 순회하며 이름-값 쌍 출력)이 빠졌습니다. 딕셔너리를 통째로 출력했을 뿐입니다.

③ 추가 + 안전장치

```
sensor_name = ["온도", "진동", "압력"]
sensor_val = [92.4, 4.8, 12.4]
sensors = dict(zip(sensor_name, sensor_val))
print(f"zip 결과: {sensors}")

# ③ items 순회 출력이 없음
for name, value in sensors.items():
    print(f" {name}: {value}")

# 안전장치를 하나 더 — STEP 6에서 본 개수 불일치 방지
print("개수 일치:", len(sensor_name) == len(sensor_val))
```

- ▶ zip 결과: {'온도': 92.4, '진동': 4.8, '압력': 12.4}
- ▶ 온도: 92.4
- ▶ 진동: 4.8
- ▶ 압력: 12.4
- ▶ 개수 일치: True

실습 1~4 요약

실습	핵심 확인 사항	결과	보완
1. 만들고 다루기	get·in을 둘 다 써봤는가	대체로 통과	기존 키 수정 1줄 추가
2. update 갱신	update·del·len 세 가지를 다 썼는가	③ 누락	del + len 추가
3. 통계 내기	items로 합계와 최댓값을 한 번에	통과	초기값 타입·범위 정리
4. zip 매핑	zip 결과를 items로 다시 풀었는가	③ 누락	items 출력 + 개수 확인

개념

네 문제 모두 로직은 맞고 요구사항 일부가 빠진 형태입니다. 문법을 몰라서가 아니라 요구사항 ①②③을 끝까지 따라가지 않은 것이죠. 실습 파일에 요구사항을 주석으로 적어두셨으니, 각 요구사항을 처리한 줄 옆에 # ① # ② # ③ 표시를 달아두면 빠진 항목이 눈에 바로 보입니다. 코드를 다 짰 뒤 주석 번호를 세어 보는 것만으로 누락이 잡힙니다.

STEP 10

실습 5~8

중복 버그 · 미완성 3건 완성

뒤쪽 네 문제는 난이도가 한 단계 올라갑니다. 5번은 완성했지만 **눈에 잘 안 띄는 버그**가 있고, 6번은 정확하며, 7·8번은 미완성입니다.

실습 5 — 임계값으로 경고 센서 분류하기

요구사항보다 훨씬 어렵게 푸셨습니다. 교안은 평평한 딕셔너리 두 개를 비교하라고 했는데, **딕셔너리 리스트 10건 × 임계값 11종**으로 확장해 이중 반복으로 처리하셨습니다. 구조 이해도는 충분히 보여줍니다. 그런데 출력을 세어 보면 이상합니다.

제출 코드 — 실습 5

```
warning = []
for sensor in sensors_list:
    name = sensor["센서 이름"]
    for key, value in sensor.items():
        if key == "센서 이름":
            continue
        elif key in limits and value > limits[key]:
            warning.append(name)      # ← 항목마다 append
```

```
print(f"초과 리스트:", warning)
```

```
▶ 초과 리스트: ['compressor_temp', 'bearing_vibe', 'bearing_vibe', 'fan_current',
▶                  'gearbox_noise', 'gearbox_noise', 'turbine_rpm', 'hydraulic_oil',
▶                  'hydraulic_oil', 'conveyor_load']
▶
▶ ↑ 리스트 길이 10 · 실제 설비는 7대
```

설비는 7대인데 리스트에는 10개가 들어 있습니다. **한 설비에서 두 항목이 동시에 임계값을 넘으면 이름이 두 번 담기기** 때문입니다.

설비	초과 항목	append 횟수
compressor_temp	온도 92.4 > 80.0	1회
bearing_vibe	진동 8.2 > 5.0 · 속도 1750 > 1500	2회 ✕
fan_current	전류 24.5 > 20.0	1회
gearbox_noise	진동 5.1 > 5.0 · 소음 88.5 > 85.0	2회 ✕

설비	초과 항목	append 횟수
turbine_rpm	회전수 3590 > 3500	1회
hydraulic_oil	압력 12.4 > 10.0 · 점도 46.2 > 45.0	2회 ✕
conveyor_load	하중 450 > 400	1회

자주 하는 실수

"경고가 몇 번 났나"와 "몇 대가 문제인가"는 다른 질문입니다. 지금 리스트는 전자를 세고 있고, 요구사항은 후자를 묻습니다. 리포트에 "이상 설비 10대"라고 찍히면 실제 7대보다 부풀려진 숫자가 됩니다. 어제 배운 셋을 떠올리면 바로 답이 나옵니다 — **종류가 궁금하면 셋**입니다.

고친 코드

```
warning = set()                # 리스트 → 셋
detail = []                   # 상세 근거는 따로 남긴다

for sensor in sensors_list:
    name = sensor["센서 이름"]
    for key, value in sensor.items():
        if key == "센서 이름":
            continue
        if key in limits and value > limits[key]:    # elif → if
            warning.add(name)
            detail.append(f"{name}: {key} {value} > {limits[key]}")

print(f"초과 설비 {len(warning)}대:", sorted(warning))
for d in detail:
    print("  - ", d)
```

```
▶ 초과 설비 7대: ['bearing_vibe', 'compressor_temp', 'conveyor_load', 'fan_current',
▶               'gearbox_noise', 'hydraulic_oil', 'turbine_rpm']
▶   - compressor_temp: 온도 92.4 > 80.0
▶   - bearing_vibe: 진동 8.2 > 5.0
▶   - bearing_vibe: 속도 1750 > 1500
▶   - fan_current: 전류 24.5 > 20.0
▶   - ... (총 10건)
```

개념

`elif`를 `if`로 바꾼 것도 의미가 있습니다. 바로 위 줄에서 `continue`가 흐름을 끊으므로 `elif`에 도달하는 경우는 `if`와 동일합니다. **동작은 같지만** `elif`는 "앞 조건과 배타적"이라는 잘못된 신호를 줍니다. **실제로는 독립된 검사이므로** `if`가 의도를 정확히 표현합니다. 그리고 `print(f"임계값:", limits)`는 f-string인데 중괄호가 없어 `f`가 아무 일도 하지 않습니다 — `print("임계값:", limits)`로 충분합니다.

실습 6 — 중첩 딕셔너리로 설비 관리하기

세 요구사항을 모두 정확히 처리했습니다. 특히 `.items()`로 설비명과 상세 정보를 동시에 받아 안쪽 딕셔너리의 `status`를 검사한 부분이 오늘 배운 중첩 구조의 정석입니다.

제출 코드 — 실습 6 (정확함)

```
factory_equipment = {
    "Compressor_A": {"status": "정상", "temperature": 42.5, "pressure": 3.2},
    "Pump_B": {"status": "경고", "temperature": 68.1, "pressure": 1.8},
    "Motor_C": {"status": "정상", "temperature": 35.4, "vibration": 2.5},
}

print("Pump_B의 온도:", factory_equipment["Pump_B"]["temperature"])

for name, info in factory_equipment.items():
    if info["status"] == "경고":
        print(f"상태 경고인 설비: {name}")
```

- ▶ Pump_B의 온도: 68.1
- ▶ 상태 경고인 설비: Pump_B

응용

한 걸음 더 나가면 이렇게 확장할 수 있습니다. `Motor_C`에는 `pressure` 키가 없고 `vibration`이 있습니다 — **설비마다 측정 항목이 다른** 현실적인 데이터입니다. 안쪽 키를 대괄호로 꺼내면 `KeyError`가 나므로 `info.get("pressure", -1)`로 접근해야 안전합니다. 실무 데이터는 거의 항상 이렇게 들쭉날쭉하므로, **중첩 딕셔너리의 안쪽은 `get`으로 접근하는 습관**을 들이는 게 좋습니다.

실습 7 — 표 데이터를 딕셔너리로 변환하기

데이터 형태를 잘못 잡으셨습니다. 요구사항 ①은 "센서명, 측정값" 형태의 행 문자열들을 **리스트로 저장하는 것인데**, 이름과 값이 번갈아 들어간 **평평한 리스트**로 만드셨습니다.

	작성한 데이터	요구사항의 데이터
형태	["온도", 78.5, "진동", 4.8]	["온도,78.5", "진동,4.8"]
요소 하나	이름 또는 값 (번갈아)	한 행 전체가 문자열 하나
다음 단계	짝수·홀수 인덱스로 갈라야 함	split(",")로 나눈다

요구사항이 이 형태를 요구한 이유는 **CSV 파일이 실제로 이렇게 생겼기 때문**입니다. 파일 한 줄을 읽으면 "온도,78.5"라는 문자열 하나가 통째로 들어옵니다. 5일차에 배운 `split()`이 여기서 쓰입니다.

완성본

```
rows = ["온도,78.5", "압력,95.0", "진동,0.5"]    # ① 행 문자열 리스트

result = {}
for row in rows:
    name, value = row.split(",")                # ② 쉼표로 나눠 언패킹
    result[name] = float(value)                 # ③ 값은 숫자로 변환해 저장

print(result)
print("온도만:", result["온도"], type(result["온도"]).__name__)
```

```
▶ {'온도': 78.5, '압력': 95.0, '진동': 0.5}
▶ 온도만: 78.5 float
```

개념

`row.split(",")`는 ["온도", "78.5"]라는 **리스트**를 돌려주고, `name, value = ...`가 그것을 두 변수로 언패킹합니다. 어제 배운 "리스트도 언패킹된다"가 여기서 쓰입니다. 그리고 `float(value)`가 필수입니다 — `split` 결과는 **전부 문자열**이라 "78.5" 그대로 두면 나중에 더하거나 비교할 때 문제가 됩니다. 4일차의 "입력받은 값은 무조건 str"과 같은 맥락입니다.

실습 8 — 센서 데이터 통합 정리

데이터만 붙여넣고 로직이 없습니다. 요구사항 네 개를 보면 **오늘 배운 것과 어제 배운 것을 합치는 문제**입니다 — `values`로 평균, `items`로 순회, 초과 센서를 셋에 모아 정렬 출력.

완성본 — 교안 예상 출력과 일치

```
values = {"온도": 95, "압력": 96}          # ① 측정값
limits = {"온도": 90, "압력": 90}         # ① 임계값

avg = sum(values.values()) / len(values)  # ② values로 평균
print(f"평균: {round(avg, 1)}")

danger = set()                            # ③ 셋에 모은다
for name, value in values.items():
    if value > limits.get(name, float("inf")):
        danger.add(name)

print("위험센서:", sorted(danger))        # ④ 정렬 출력
```

▶ 평균: 95.5

▶ 위험센서: ['압력', '온도']

실습 5에서 쓰신 확장 데이터(딕셔너리 리스트 10건)로 풀면 이렇게 됩니다. 실습 5의 고친 코드에 평균 계산만 없으면 됩니다.

확장 데이터 버전

```
danger = set()
total = 0
count = 0

for sensor in sensors_list:
    name = sensor["센서 이름"]
    for key, value in sensor.items():
        if key == "센서 이름":
            continue
        total += value                # 이름 칸을 뺐으니 전부 숫자
        count += 1
        if value > limits.get(key, float("inf")):
            danger.add(name)

print(f"전체 측정 항목 {count}개 · 평균 {round(total / count, 1)}")
print(f"위험 설비 {len(danger)}대:", sorted(danger))
```

▶ 전체 측정 항목 20개 · 평균 336.6

▶ 위험 설비 7대: ['bearing_vibe', 'compressor_temp', 'conveyor_load', 'fan_current',
'gearbox_noise', 'hydraulic_oil', 'turbine_rpm']

주의

확장 버전의 평균 336.6은 의미가 없는 숫자입니다. 온도(°C) · 진동(mm/s) · 회전수(rpm) · 전압(V)처럼 단위가 다른 값을 한 통에 넣고 평균을 냈기 때문입니다. 회전수 3590 하나가 전체 평균을 끌어올린 결과죠. 교안 데이터가 온도 · 압력만 담은 이유가 여기 있습니다. 통계를 내기 전에 "같은 단위인가"를 먼저 확인하는 습관이 필요합니다 — 이걸 문법 문제가 아니라 데이터 분석의 기본입니다.

실습 8건 자가 점검표

실습	요구사항	결과
1. 만들고 다루기	생성 · 조회/추가/수정 · get·in	수정 1줄 누락
2. update 갱신	update · del · len	del·len 누락
3. 통계 내기	values 평균 · items 최댓값	통과 (초기값 개선 권장)
4. zip 매핑	zip · dict · items 출력	items 출력 누락
5. 임계값 분류	items 비교 · 리스트에 모으기	중복 3건 발생
6. 중첩 딕셔너리	중첩 조회 · items로 경고 검색	통과
7. 표 데이터 변환	행 문자열 · split · 숫자 변환	미완성
8. 통합 정리	평균 · 셋에 모으기 · 정렬	미완성

치트시트

딕셔너리 한 장 요약

만들기 · 꺼내기

	문법	하는 일	없는 키일 때
문법	<code>{"키": 값}</code>	딕셔너리 생성	—
내장함수	<code>dict(zip(키들, 값들))</code>	두 리스트를 짝지어 생성	짧은 쪽에 맞춰 잘림
문법	<code>d["키"]</code>	값 꺼내기	KeyError
메서드	<code>d.get("키")</code>	값 꺼내기 (안전)	<code>None</code>
메서드	<code>d.get("키", 기본값)</code>	값 꺼내기 (기본값 지정)	기본값
연산자	<code>"키" in d</code>	키 존재 확인	<code>False</code>
문법	<code>d["A"]["B"]</code>	중첩 값 꺼내기	KeyError

고치기

	문법	하는 일	주의
문법	<code>d["새키"] = 값</code>	추가	있는 키면 수정이 된다
문법	<code>d["있는키"] = 값</code>	수정	추가와 문법이 같다
키워드	<code>del d["키"]</code>	키-값 통째로 삭제	없는 키면 KeyError
메서드	<code>d.update(다른딕셔너리)</code>	여러 쌍 한 번에 반영	겹치는 키를 조용히 덮어씀

순회하기

코드	한 번에 나오는 것	변수	대표 용도
<code>for k in d:</code>	키 하나	1	센서 이름 목록 훑기
<code>for k in d.keys():</code>	키 하나	1	위와 동일 · 의도가 더 명확
<code>for v in d.values():</code>	값 하나	1	합계 · 평균 · 최댓값

코드	한 번에 나오는 것	변수	대표 용도
<code>for k, v in d.items():</code>	(키, 값) 튜플	2	이름과 값을 같이 — 최다 사용

내장 함수와 조합

코드	결과	비고
<code>len(d)</code>	키-값 쌍의 개수	값의 개수와 같다
<code>sum(d.values())</code>	값들의 합	값이 전부 숫자여야 함
<code>max(d.values())</code>	가장 큰 값	이름은 못 얻는다 → items 필요
<code>sorted(d)</code>	키를 정렬한 리스트	<code>sorted(d.keys())</code> 와 같음
<code>list(d.items())</code>	(키, 값) 튜플의 리스트	어제 배운 튜플 리스트 그 자체
<code>set(d.keys())</code>	키들의 셋	집합 연산으로 두 딕셔너리 비교

자주 쓰는 패턴 4가지

외워둘 네 덩어리

```
# ① 빈 딕셔너리에 채워 넣기
readings = {}
for name, value in raw_list:
    readings[name] = value

# ② 두 딕셔너리 대조 (임계값 판정)
for name, value in values.items():
    if value > limits.get(name, float("inf")):
        print(name, "경고")

# ③ 조건에 맞는 키를 셋에 모으기 (중복 방지)
danger = set()
for name, value in values.items():
    if value > LIMIT:
        danger.add(name)

# ④ 딕셔너리 리스트 순회 (실무 최다)
for row in records:
    name = row["센서 이름"]
    for key, value in row.items():
        if key == "센서 이름":
            continue
    ...
```

오류·함정 사전

한 번씩은 반드시 겪는 것들

B-1 · 오늘 실제로 겪은 것

✗ 이렇게 하면	무슨 일이	✓ 바로잡기
<code>limits.get(name, 0)</code>	임계값 없는 센서가 전부 경고 (오경보)	<code>limits.get(name, float("inf"))</code>
리스트에 append로 모음	한 설비가 여러 번 초과하면 중복 집계	셋에 add로 모은다
<code>for k, v in d:</code>	too many values to unpack (expected 2)	<code>for k, v in d.items():</code>
<code>top_val_name = 0</code>	문자열 자리에 숫자 — 빈 데이터면 0 출력	<code>top_val_name = ""</code>
<code>top_val = 0</code>	음수만 있으면 최댓값을 못 찾음	<code>float("-inf")</code> 또는 첫 값
단위 다른 값을 함께 평균	숫자는 나오지만 의미가 없음	같은 단위끼리만 통계
<code>print(f"임계값:", limits)</code>	종괄호가 없어 f가 아무 일도 안 함	<code>print("임계값:", limits)</code>

B-2 · 딕셔너리 문법 함정

✗ 이렇게 하면	무슨 일이	✓ 바로잡기
<code>d["없는키"]</code>	KeyError — 그 자리에서 실행 중단	<code>d.get("없는키", 기본값)</code>
<code>del d["없는키"]</code>	KeyError	<code>if "키" in d: del d["키"]</code>
<code>{"a", "b": 1}</code>	TypeError: unhashable type: 'list'	키는 문자열·숫자·튜플만
<code>d = d.update(x)</code>	update는 None 반환 → 딕셔너리가 사라짐	<code>d.update(x)</code>
<code>0.5 in d</code>	값이 아니라 키를 찾아서 False	<code>0.5 in d.values()</code>
<code>sum(d.values())</code>	값에 문자열이 섞여 있으면 TypeError	숫자 항목만 골라 합산
<code>empty = {}</code>	셋이 아니라 딕셔너리가 생성됨	빈 셋은 <code>set()</code> — 어제 배운 함정
<code>d.len()</code>	AttributeError — len은 메서드가 아님	<code>len(d)</code>

B-3 · zip · update 사고 사례

상황	증상	대응
zip에 키 4개 · 값 3개	오류 없이 4번째가 사라짐	<code>len(a) == len(b)</code> 먼저 확인
zip에 키 3개 · 값 4개	오류 없이 4번째 값이 버려짐	같은 확인
update로 이름 키가 겹침	이름만 바뀌고 데이터는 그대로 남음	겹치는 키를 미리 출력해 확인
두 딕셔너리 키 표기 불일치	"모터 온도" vs "모터온도" — 영원히 매칭 안 됨	키 이름 규칙을 먼저 통일

개념

부록 B에 실린 것 중 **오류 메시지가 뜨는 것은 절반뿐**입니다. 나머지 절반(오경보·중복 집계·zip 누락·update 덮어쓰기·의미 없는 평균)은 프로그램이 멀쩡히 돌면서 결과만 틀립니다. 어제와 같은 결론입니다 — **예상 출력을 먼저 적어두고 실제 출력과 대조하는 것이 유일한 방어책**입니다.

실습 하이라이트 & 다음 시간 예고

기초 자료형의 끝, 그리고 다음

C-1 · 오늘 작성한 파일

파일	내용	상태
Python/16_dictionary.py	딕셔너리 생성 · 딕셔너리 리스트 · get · items 임계값 비교	수업 코드 · 오경보 1건
Practice/06_02_example.py	포켓몬 워밍업 + 교안 실습 1~8	6건 완료 · 2건 미완성
Lecture/06_02_딕셔너리.pdf	PART 1 기초 · PART 2 순회와 갱신 · PART 3 실습 8개	전 범위 진도 완료

C-2 · 오늘 배운 것 한눈에

분류	오늘 나온 것	개수
메서드	get · keys · values · items · update	5개
키워드	del	1개
연산자	in	1개
내장 함수	dict · zip · len · sum · sorted	5개
문법	{키: 값} · d[키] · d[키] = 값 · d[A][B]	4개
관례	스네이크 케이스 · 대문자 상수	2개

주의

교안 06_02에 나오는 딕셔너리 메서드는 이 다섯 개가 전부입니다.

pop · clear · copy · setdefault · popitem · fromkeys는 등장하지 않습니다. 헛갈리기 쉬운 것은 메서드가 아닌데 함께 나온 것들입니다 — del은 키워드, in은 연산자, len · zip은 내장 함수, d[키] = 값은 문법입니다. 점(.)이 찍혀 있는지만 보면 3초 안에 갑니다.

C-3 · 자료형 4형제 완주

오늘로 파이썬의 기본 자료구조를 전부 배웠습니다. 지금까지의 여정을 한 줄씩 정리하면 이렇습니다.

배운 날	자료형	한 줄 정의	대표 문법
6일차	리스트	순서대로 쌓고 자유롭게 고친다	<code>lst.append(x)</code>
8~9일차	튜플	한 번 정한 짝을 지킨다	<code>name, value = t</code>
9일차	셋	중복을 지우고 목록을 비교한다	<code>a & b</code> · <code>a - b</code>
10일차	딕셔너리	이름표로 값을 찾는다	<code>for k, v in d.items()</code>

C-4 · 다음 시간 예고

다음 주제	한 줄 미리보기
함수 (def)	반복되는 코드에 이름을 붙여 재사용. 강사님이 "함수를 배울 때 튜플이 왜 필요한지 다시 설명하겠다"고 예고하셨다
sorted의 key=	정렬 기준을 직접 지정. 함수를 배워야 풀리는 숙제로 남겨둔 것
파일 읽기	CSV를 읽어 split으로 나누고 딕셔너리로 — 실습 7이 그 예고편이었다
pandas	오늘의 딕셔너리 리스트가 DataFrame이 된다. 표 형태 데이터 분석의 시작

연결

강사님이 남긴 예고 두 가지를 기억해 두면 좋습니다. 첫째, 튜플은 함수를 배울 때 다시 나옵니다 — 지난주 금요일에 급하게 지나간 부분을 함수와 함께 다시 설명하시겠다고 했습니다. 둘째, key= 정렬은 함수를 알아야 풀립니다 — 어제 미뤄둔 숙제가 다음 단계에서 해결됩니다. 지금이 파이썬 기초의 마지막 구간이고, 여기를 넘으면 실제 데이터 분석 도구로 넘어갑니다.

오늘의 성취와 다음 한 걸음

번호에서 이름으로

오늘 배운 것을 한 문장으로 줄이면 이렇습니다 — 데이터에 이름표를 붙여 저장하고, `items()`로 이름과 값을 함께 꺼내 처리한다. 리스트가 "몇 번째"만 알던 것에 비해, 딕셔너리는 데이터가 스스로를 설명합니다. 설비 데이터처럼 이름이 분명한 정보에는 이쪽이 훨씬 자연스럽습니다.

그리고 오늘로 리스트·튜플·셋·딕셔너리 네 개가 모두 손에 들어왔습니다. 앞으로 마주칠 어떤 데이터든 이 넷 중 하나, 혹은 이들을 겹친 구조로 표현할 수 있습니다. 실제로 오늘 마지막에 배운 `[{...}, {...}]`가 그 겹침의 첫 사례였습니다.

오늘 헛갈렸던 지점 다시 짚기

헛갈린 것	정리
메서드인가 아닌가	점(.)이 있으면 메서드 — <code>get·keys·values·items·update</code> 다섯 개뿐. <code>del·in·len·zip</code> 은 아니다
<code>items</code> 를 왜 붙이나	딕셔너리를 그냥 순회하면 키 하나만 나온다. 변수 2개로 받으려면 2개짜리를 주는 <code>items()</code> 가 필요
<code>[]</code> 와 <code>get</code> 차이	있는 키면 똑같다. 없는 키에서 갈린다 — 대괄호는 <code>KeyError</code> 로 멈추고, <code>get</code> 은 <code>None</code> (또는 기본값)으로 넘어간다
추가와 수정	둘 다 <code>d[키] = 값</code> . 없으면 추가, 있으면 덮어쓰기. 리스트처럼 <code>append</code> /수정이 나뉘지 않는다
기본값 뭘로 두나	합산이면 <code>0</code> , 결측 표시면 <code>-1</code> , 임계값이면 <code>float("inf")</code> . 습관적으로 <code>0</code> 을 쓰면 오경보가 난다
셋인가 딕셔너리인가	같은 종괄호지만 키: 값이면 딕셔너리, 값만 나열하면 셋. 빈 <code>{}</code> 는 딕셔너리

개념

복습 루틴 추천: ① 메서드 5개를 점 유무로 판별하는 연습 → ② `for k, v in d.items():` 를 눈 감고 쓸 수 있을 때까지 → ③ `d[키]`와 `d.get(키)`를 **없는 키로 각각 실행**해 차이를 눈으로 확인 → ④ `16_dictionary.py`의 `limits.get(name, 0)`을 `float("inf")`로 바꿔 출력이 어떻게 달라지는지 직접 확인 → ⑤ 미완성인 실습 7·8을 STEP 10을 덮고 처음부터 풀어보기. ④는 오늘 문서에서 가장 중요한 대목이라 **코드로 직접 확인**해 보시길 권합니다.